Data Engineering Interview Training Notes – Page 1

Mental model, ingestion, data quality, duplicates, and real-world decision making

1. The Mental Model

Source Systems
- DB / OLTP (e.g. MySQL, PostgreSQL, SQL Server)
- Files (CSV, JSON, etc.)
- APIs (REST, GraphQL)
- Event streams (Kafka, Kinesis, etc.)
- SaaS apps (e.g. CRM, ERP)

→ **Ingestion**
- Full load
- Incremental load
- CDC (Change Data Capture)
- API / file ingestion
- Streaming ingestion

→ **Raw / Bronze**
- Keep source truth
- Minimal transforms
- Partitioned storage (e.g. by date)
- Append / immutable where possible

→ **Validation & Cleaning**
- Schema checks
- Data quality checks
- Deduplication
- Standardization
- Quarantine bad records

→ **Curated / Silver**
- Cleaned data
- Conformed schema
- Business logic
- Standardized formats (derived fields, etc.)

→ **Data Modeling / Gold**
- Facts and dimensions
- Aggregations
- Optimized for consumption
- Business ready data

→ **Consumption**
- BI dashboards (e.g. Power BI, Tableau)
- Data science / ML
- Reverse ETL
- Operational apps (internal tools, apps, etc.)

★ Key interview-thinking questions (always ask these!)

1. What is the source system?
2. Batch or streaming, and why?
3. Expected data volume?
4. What is the change pattern (insert / update / delete)?
5. What is the business key?
6. What is the SLA/latency?
7. Do we need historical tracking?
8. How are deletes handled?
9. Can late-arriving data occur?
10. What data quality rules apply?

2. Ingestion Types

Type	When to use	Pros	Cons	Example
Full Load	Small to medium data or initial load	Simple, complete data	Higher load time, more resource usage	Load all customers table once a day
Incremental / Watermark	Large tables with updated records	Faster, lower cost	Need reliable timestamp / watermark	Fetch records where updated_at > last_ts
CDC (Change Data Capture)	Databases with frequent changes	Near real-time, only changed data	More complex setup	Use DB logs (e.g. Debezium)
API Ingestion	Data from external systems (SaaS/3rd party)	Flexible, real-time or batch	Rate limits, retries, pagination	REST API with pagination
File Ingestion	Data provided as files (CSV, JSON, etc.)	Simple, widely used	File arrival delays, duplicate files	Daily CSV file via SFTP
Streaming / Event Ingestion	Real-time data needs	Low latency,	Higher complexity, cost	Kafka / Kinesis events

3. Watermark & Incremental Logic

- Maintain last successful extraction timestamp.
- Use overlap window to avoid missing late data.
- Store watermark in a control table.
- Safer to process a small duplicate than to miss data.

Pseudo-SQL (basic):

```
SELECT * FROM source_table
  WHERE updated_at > :last_success_ts;
```

With overlap window (safer):

```
SELECT * FROM source_table
  WHERE updated_at >= :last_success_ts
    - INTERVAL '5 minutes';
```

Control table example:

pipeline_name	last_success_ts	status	remarks
orders	2024-01-10 10:00:00	SUCCESS	12000 records
customers	2024-01-10 09:45:00	SUCCESS	5000 records
products	2024-01-10 09:30:00	FAILED	API timeout

4. Batch vs Streaming

Batch Processing
- Processes data in chunks (at scheduled intervals)
- Higher latency (minutes to hours)
- Simpler architecture
- Cost effective
- Good for large volumes

Examples:
- Daily / hourly ETL jobs
- Generate daily reports

Streaming Processing
- Processes data in real time or near real time
- Low latency (seconds to minutes)
- More complex architecture
- Higher cost
- Needed for time-sensitive use cases

Examples:
- Real-time fraud detection
- Live dashboards, real-time alerts

Decision rule: choose using **business SLA**, not hype.
- If data is needed in seconds or minutes → Streaming
- If hourly or daily is okay → Batch
- Also consider: data volume, cost, complexity, and business value.

5. Handling Duplicate Data

Common reasons for duplicates:
1. Source system replays
2. Same file resent
3. Retries during failures
4. CDC reprocessing / log replays
5. Late arriving events
6. Overlapping watermark windows

Common dedup strategies:
1. Unique business key + latest updated_at
2. ROW_NUMBER window function
3. Hash comparison for full-row checks
4. File-level idempotency
5. Event id dedup for streaming

Example data with duplicates:

business_id	name	updated_at	amount
101	Alice	2024-01-10 10:00:00	100
101	Alice	2024-01-10 11:00:00	150
102	Bob	2024-01-10 09:30:00	200

Deduplicate using ROW_NUMBER():

```
SELECT * FROM (
  SELECT *,
    ROW_NUMBER() OVER (PARTITION BY business_id
                       ORDER BY updated_at DESC) as rn
  FROM source_table
) WHERE rn = 1;
```

6. Data Quality Checks during Ingestion

Checklist:
- ☐ Schema validation (expected columns, types)
- ☐ Null checks (required fields)
- ☐ Data type checks (e.g. string, int, date)
- ☐ Primary / business key uniqueness
- ☐ Referential checks (valid foreign keys)
- ☐ Record-count reconciliation (source vs target)
- ☐ Duplicats checks
- ☐ Freshness checks (data not too old)
- ☐ Range checks / business rule checks (e.g. amount > 0)

Bad Records Handling:
- Send invalid records to quarantine / reject table.
- Capture rejection reason (e.g. null in required field, invalid format).
- Monitor and report (alerts, dashboards).
- Fix data and reprocess if needed.
- Keep rejected data for audit and debugging.

7. Talking points while teaching

1. Ask what makes a record unique (business key).
2. Better to reprocess than miss data.
3. Raw layer preserves source truth.
4. Quality checks belong early in the pipeline.
5. Do not assume source is clean.
6. Architecture follows requirements, not buzzwords.
7. Always discuss business key + SLA + history + deletes.

Data Engineering Interview Training Notes – Page 2

data modeling, schema choice, SCDs, performance, orchestration, and reliability

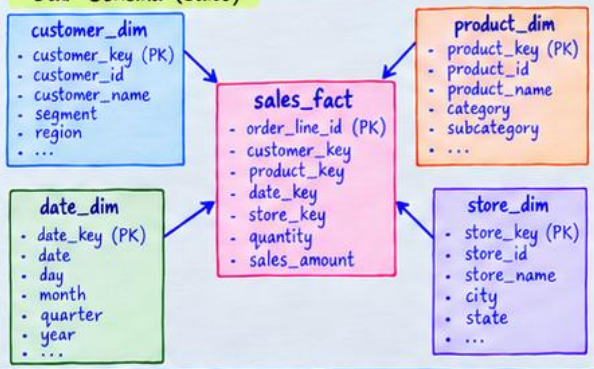
1. Data Modeling Basics

- **Fact table**: stores measurable events (e.g. sales, orders) at a defined grain.
- **Dimension table**: stores descriptive attributes (e.g. customer, product).
- **Grain**: level of detail of a fact table (e.g. one row per order line).
- **Measures**: numeric values that can be aggregated (e.g. quantity, revenue).
- **Surrogate key**: artificial key (e.g. integer) used to join tables.
- **Natural / Business key**: real world key (e.g. customer_id, product_id).

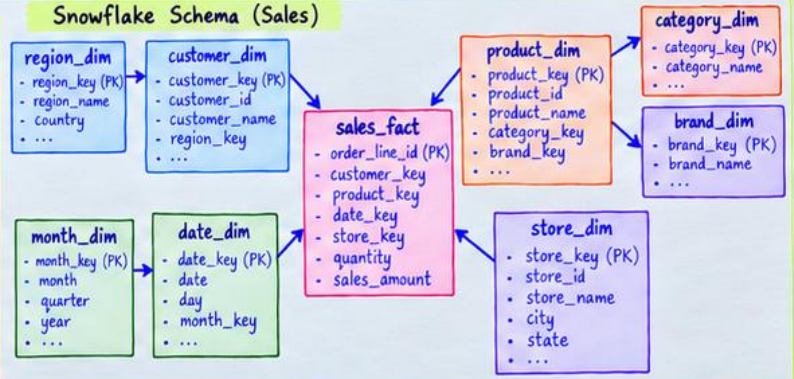
★ **Define grain first!**
e.g. one row per order line

2. Star Schema vs Snowflake Schema

Star Schema (Sales)



Snowflake Schema (Sales)



Schema Type	What it is	When to use	Pros	Cons
Star Schema	Fact table + denormalized dimensions	Most common for BI/analytics	• Simple and easy to query • Good performance • Works well with BI tools	• Redundant data • More storage • Not ideal for very complex hierarchies
Snowflake Schema	Normalized dimensions (split into multiple tables)	When dimensions have complex hierarchies	• Less redundant data • Better data integrity	• More joins • Complex queries • Slower performance
One Big Table / Wide Table	Single denormalized table with all data	Small datasets or simple use cases	• Very simple • No joins • Fast for ad-hoc analysis	• Not scalable • Lots of nulls • Hard to maintain • Not good for complex logic
Data Vault	Hub, Link, Satellite tables (normalized, history tracking)	Large, enterprise systems with changing data	• Handles change well • Scalable and flexible • Good for multiple sources • Full history	• Complex to implement • More tables and joins • Requires expertise • Not ideal for simple use cases

3. Slowly Changing Dimensions (SCDs)

Type	What it is	When to use	Pros	Cons
Type 1 (Overwrite)	Overwrite old value with new value	When history is not required	• Simple • No change in table size	• No history of changes • Old values lost
Type 2 (New row)	Insert new row with effective dates and current flag	When history needs to be maintained	• Full history of changes • Auditable	• Larger table size • More complex queries
Type 3 (Additional column)	Keep current and previous value in same row	When only limited history (e.g. last value) is needed	• Simple • Easy to query • No joins	• Only tracks limited history • Not suitable for multiple changes

Example: Customer city change (Type 2)

customer_key	customer_id	customer_name	city	effective_from	effective_to	current_flag
101	C001	Alice	Delhi	2023-01-01	2024-06-30	N
102	C001	Alice	Mumbai	2024-07-01	9999-12-31	Y

When to choose which?

- **Type 1**: no history needed (e.g. fix errors, simple attributes)
- **Type 2**: full history needed (e.g. customer address, product attributes)
- **Type 3**: only current vs previous value needed (e.g. current status)

5. Partitioning, File Formats & Performance

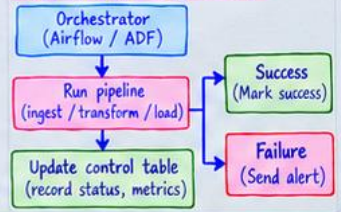
- **Partitioning**: split data by date (e.g. year, month, day) or low-cardinality fields (e.g. country, region) to improve query performance and reduce scan size.
- **Small files problem**: many small files slow down queries and increase metadata cost.
- **Compaction**: combine small files into larger files (e.g. 128MB – 1GB) for better performance.

Format	What it is	Pros	Cons	Best for
Parquet	Columnar storage format	• Columnar (better performance) • Compressed • Supports schema evolution	• Not human readable • Slightly complex	Analytical workloads, large datasets
CSV	Plain text, row format	• Simple • Human readable • Widely supported	• Larger file size • No schema • Slower performance	Small data, data exchange
JSON	Semi-structured text format	• Flexible schema • Human readable • Works for nested data	• Larger file size • Slower performance • Not most efficient	Logs, semi-structured data, APIs

6. Orchestration, Idempotency & Monitoring

- **Orchestration tools**: Airflow, AWS Step Functions, Azure Data Factory, etc. to schedule and manage data pipelines.
- **Retries and error handling**: configure retries for transient failures.
- **Dependencies**: define task dependencies (e.g. ingest → transform → load).
- **Idempotent pipelines**: running the same pipeline multiple times should produce the same result (no duplicates).
- **Audit / control tables**: store run_id, status, start_time, end_time, records processed, etc.
- **Monitoring and alerting**: track pipeline runs, data delays, failures, and SLA breaches.

Pipeline Flow (High Level)



7. Data Lakehouse / Medallion Layers

Bronze (Raw)
- Raw data from sources
- Minimal processing
- Keep original format
- Append only
- Store ingestion metadata

→ **Silver (Cleaned)**
- Cleaned and validated
- Standardized schema
- Deduplicated
- Conformed data
- Ready for analytics

→ **Gold (Business Ready)**
- Business ready data
- Aggregations and metrics
- Optimized for consumption
- Curated for BI / reporting
- Trusted and documented

8. Talking points while teaching

1. Explain what a fact table is and what a dimension is.
2. Discuss grain and why it is important.
3. Compare star vs snowflake schema and trade-offs.
4. Explain SCD types with real examples.
5. Cover partitioning, file formats and performance tips.
6. Discuss how deletes and updates are handled.
7. Explain orchestration, idempotency and monitoring.
8. Tie everything together with an end-to-end pipeline example (Bronze → Silver → Gold).

Data Engineering Interview Training Notes – Page 3

CDC, data quality, validation frameworks, reconciliation, metadata, and ingestion edge cases

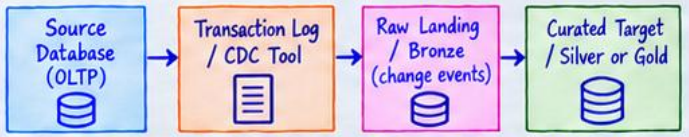
1. CDC (Change Data Capture) – What it is

- CDC captures only the changes from source systems instead of reloading the whole table.
- Captures INSERT, UPDATE and DELETE operations as they happen.
- More efficient, near real-time, reduces load on source.
- Common approaches:
  - **Log-based CDC**: reads database transaction logs (e.g. log mining, WAL, binlog).
  - **Trigger-based CDC**: uses database triggers to capture changes into a change table.
  - **Query-based incremental**: periodically queries source based on a timestamp or increasing id (e.g. updated_at > last_run).

Example: CDC events for a customer table

operation	id	name	email	event_time
I	101	Alice	alice@example.com	2024-01-10 10:00
U	101	Alice A	alice.a@example.com	2024-01-10 10:05
I	102	Bob	bob@example.com	2024-01-10 10:10
D	101	NULL	NULL	2024-01-10 10:20

2. How CDC Works in Practice



- Generates transaction logs for each change (INSERT / UPDATE / DELETE)
- Reads and parses transaction logs or uses triggers / incremental queries
- Extracts change events with metadata (e.g. operation type, LSN / offset, timestamp)
- Stores raw change events exactly as captured (append-only)
- Keeps metadata (LSN / offset, source table, event time)
- Maintains ordering within a table
- Applies changes to build current view of data
- Handles deduplication, ordering, late events,
- May keep history for auditing (SCD Type 2)

Key points:

- CDC maintains ordering using LSN / offset / timestamp.
- Use checkpoints to track the last processed LSN / offset so we can resume.
- If a failure happens, we can replay from the last checkpoint.
- CDC is usually at-least-once, so idempotency and deduplication matter.

3. CDC Design Decisions

Question	Considerations / Options
What is the primary / business key?	• Use natural business key or surrogate key. • Ensure key is stable and unique.
How to order changes?	• Use source LSN / offset, commit timestamp or sequence number. • Ensure total ordering within a table.
Late / out-of-order events?	• Use event time + watermark. • Allow small delay window and reorder.
How to handle updates?	• Capture before/after values if available. • Use upsert/merge logic in target.
How to handle deletes?	• Hard delete vs soft delete (tombstone). • Keep delete records for auditing.
Snapshot + CDC bootstrap?	• Initial full load to get existing data. • Then start CDC from a consistent point.
Schema evolution?	• Handle added/removed columns. • Use schema-on-read in raw layer.
Exactly-once vs at-least-once?	• Most CDC is at-least-once. • Design idempotent merges and dedup logic.
How to reprocess safely?	• Keep raw change events and checkpoints. • Support reprocessing from a given offset/time.

4. Data Quality Dimensions

Dimension	What it means	Examples / Checks
Completeness	Required data is present and not missing.	• No nulls in mandatory fields, expected record count.
Validity	Data follows defined format, type and allowed values.	• Correct data types, valid codes, email format, date format.
Uniqueness	No unexpected duplicates for business keys.	• Unique constraints, duplicate count checks.
Consistency	Data is consistent across systems and within the dataset.	• Same value across related tables, consistent logic and formats.
Timeliness / Freshness	Data is available within expected time.	• SLA checks, data delay, last updated timestamp.
Integrity	Maintains referential integrity and business rules.	• Valid foreign keys, parent record exists, domain constraints.
Accuracy	Data correctly represents the real-world value.	• Often requires business validation or comparison with source system.

5. Where to Apply Data Quality Checks

Layer	What to check	Examples
Source (Operational)	• Source system health • Data contract / schema • Expected volume	• Table/endpoint available • Columns and data types • Expected record count / range
Ingestion (File / API / Streaming)	• File arrival and format • File completeness • Basic schema validation	• File received on time • File size > 0, correct format (CSV, JSON, etc.) • Required columns present
Bronze (Raw Landing)	• Schema validation • Data type checks • Capture all records (no loss) • Basic null / format checks	• Columns match expected • Correct data types • Record count matches source • Fewer possible
Silver (Cleansed)	• Deduplication checks • Standardization • Business rules • Conformity across sources	• Unique business keys • Standardized formats • Valid code values • Consistent data across tables
Gold (Business Ready)	• Final business rule checks • Reconciliation with source • Aggregations and metrics • Data freshness / SLA checks	• Row counts and sums match • KPIs are correct • No unexpected duplicates • Data available within SLA

6. Reconciliation & Control Checks

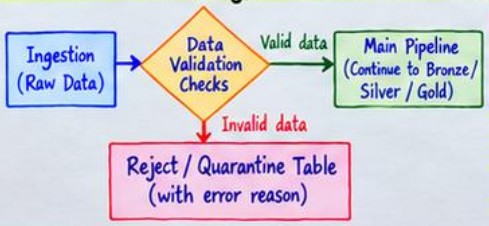
Compare source vs target to ensure data is complete and correct.

Check Type	What to check / Example
Row count check	• Compare total records between source and target (allowing for known differences).
Sum / Hash totals	• Compare sum of key numeric columns or hash of important fields (e.g. MD5 of concatenated columns).
Source vs Target reconciliation	• Match counts and key aggregates for the same time window (e.g. daily).
File count check	• Expected number of files vs actual files processed.
Duplicate count	• Check for unexpected duplicates in target.
Null count check	• Validate critical columns do not have high null counts.
Balancing checks	• e.g. total debits = total credits, or all records accounted across tables.

Example: Pipeline control / reconciliation table

pipeline_name	run_id	source_count	target_count	duplicate_count	status
orders	2024-01-10_01	100000	100000	0	SUCCESS
customers	2024-01-10_01	50000	49998	2	SUCCESS
products	2024-01-10_01	25000	25000	0	SUCCESS
transactions	2024-01-10_01	200000	199500	0	FAILED

7. Bad Data Handling, Quarantine & Observability



Key components:

- Reject / quarantine tables for bad records.
- Capture detailed error reason (e.g. invalid format, missing key, referential integrity failure).
- Set up alerts for high reject count or pipeline failures.
- Maintain audit trail for data changes and reprocessing.
- Monitor data freshness and ingestion delays.
- Track data drift (sudden changes in volume or distribution).
- Define and monitor pipeline SLAs.

8. Common ingestion edge cases + talking points

Edge case	Description	How to handle / talking points
Late-arriving data	Records arrive after expected window (e.g. due to delays in source).	• Use event time, watermark, allow late arrivals, and reprocess affected partitions.
Duplicate files	Same file ingested multiple times.	• Use file name or checksum, maintain file history, make pipeline idempotent.
Partial files	Incomplete data due to upstream issues.	• Validate file size/record count; use file validation checks; retry ingestion.
Source resend	Source resends historical data.	• Use idempotent logic, dedup based on business key and timestamp.
Missing watermark	No clear last_processed value.	• Start with a safe full or lookback load, then set watermark, maintain control table.
Schema drift	New/removed/changed columns.	• Use schema-on-read, maintain schema versions, communicate.
Null business key	Records with missing or null key.	• Reject to quarantine, capture error reason, discuss with source team.
Backfill needs	Need to reprocess historical data.	• Use parameterized backfill, run for specific range, monitor impact on downstream tables.

9. How to talk in interview

1. Explain your assumptions (e.g. source type, volume, latency, business rules).
2. Always preserve raw data (even if bad).
3. Prefer safe reprocessing over risky in-place updates.
4. Mention control tables, checkpoints and monitoring.
5. Discuss idempotency, deduplication and reprocessing strategy.
6. Talk about data quality.

Data Engineering Interview Training Notes - Page 4

AI in data engineering, evolution of the role, modern skills, AI-ready systems and practical talking points

1. Evolution of the Data Engineer Role

Traditional ETL / DW
- Batch ETL processes
- Data warehouse
- BI reporting
- Static dashboards

→ **Cloud Data Engineering**
- Scalable storage and compute
- ELT (vs ETL)
- Orchestration and automation
- Democratized data access

→ **Lakehouse / Streaming**
- Real-time pipelines
- Medallion layers (bronze, silver, gold)
- Unified analytics
- Batch + streaming together

→ **Data Products / DataOps**
- Ownership of datasets
- SLAs and observability
- Reusable / discoverable data products
- Self-serve data access

→ **AI-ready Data Platforms**
- Unstructured data (docs, images, etc.)
- Vector search
- LLM applications
- Governance and security for AI data

★ Role is no longer only moving data; it also enables trustworthy data products and AI use cases.

2. What a Modern Data Engineer Should Know

Core technical skills	Platform / business skills
• SQL (advanced querying, optimization) • Python (data processing, automation) • Spark / Distributed processing • Data modeling (relational & analytical) • Ingestion patterns (batch, streaming, CDC) • Batch + streaming data pipelines • Orchestration (Airflow, etc.) • File formats (Parquet, Avro, JSON, etc.) • Cloud fundamentals (AWS, Azure, GCP) • APIs (REST, data integration) • CDC (change data capture) • Data quality (validation, testing) • Performance tuning • Debugging and troubleshooting • CI/CD basics (pipelines, automated tests) • Git (version control)	• Business understaading (use cases, KPIs) • SLA thinking (reliability, latency, freshness) • Monitoring and observability • Cost awareness and optimization • Governance and security concepts • Documentation (designs, runbooks, data docs) • Stakeholder communication (end-to-end) • Ownership mindset • Testing and idempotency • Metadata & lineage (catalogs, documentation) • Data contracts (SLAs, schemas, ownership)

★ Depth in fundamentals matters more than knowing every tool.

3. Where AI Helps Data Engineers

Use case	How AI helps
Code assistance	Generate / complete SQL, PySpark, pipeline scaffolding
Schema mapping	Map source fields to target model (suggest transformations)
Data quality rules	Suggest validation checks and anomaly patterns
Documentation	Generate table docs, lineage summaries, runbooks
Metadata search	Natural language search over catalog / schemas
Incident support	Summarize failures, likely root causes, recovery steps
Log analysis	Detect recurring patterns in pipeline failures
Test generation	Propose unit tests and edge-case scenarios

4. Building AI-Enabled Data Systems

Sources (PDFs, docs, tickets, DB tables, APIs)

→ **Ingestion**
- Batch or streaming
- File, API, DB connectors
- Track source metadata

→ **Raw storage / Bronze**
- Store original files
- Versioned storage
- Keep raw copies

→ **Parsing / Chunking / Cleaning**
- Extract text
- Chunk documents
- Clean and normalize

→ **Embeddings**
- Create embeddings
- Model selection
- Store with metadata

→ **Vector DB / Index**
- Store embeddings
- Index and search
- Metadata filters

→ **Retrieval**
- Similarity search
- Hybrid search
- Reranking

→ **LLM / Application**
- RAG application
- Generate answers
- Use citations

→ **User / API**
- End users
- Internal apps
- APIs / tools

- Data engineers ingest both structured and unstructured data.
- Need versioning of source files, chunks and embeddings.
- Need metadata such as source, timestamp, owner, permissions.
- Keep raw copies for reprocessing.

5. What DE handles in AI systems

Data pipeline work	Reliability work	Quality work	Platform work	Governance work
• Document ingestion (PDFs, docs, tickets) • Parsing and text extraction • Chunking • Metadata extraction • Embedding pipelines • Refresh / re-embedding	• Retries and recovery • Idempotency • Dead-letter / quarantine • Backfills • Monitoring and alerting	• Chunk quality (right size, context) • Retrieval quality • Freshness of data • Duplicate documents • Source permissions • Access validation	• Vector store choice (e.g. OpenSearch, Pinecone, etc.) • Batch vs streaming refresh • Latency vs cost • Caching strategies	• PII masking / redaction • Access control (ACLs) • Auditability • Source-of-truth tracking • Compliance requirements

6. RAG / AI Data Pipeline Design Decisions

Question	Typical considerations
How often does data change?	Full rebuild vs incremental refresh (changed files only).
What is the document unit?	File, page, section, ticket, record, etc.
Chunking strategy?	Fixed size (e.g. 500 tokens) vs semantic chunks, overlap.
Embedding refresh?	Re-embed only changed content when possible.
Metadata?	Source, path, timestamp, owner, tags, access level.
Retrieval quality?	Filters (metadata), hybrid search, reranking, evaluation.
Access control?	Enforce user permissions during retrieval.
Cost & latency?	Choose embedding / LLM model, caching, batch jobs, async processing.
Hallucination risk?	Grounding with source data, citations, confidence checks, prompt design.
Evaluation?	Precision / recall style retrieval checks, answer quality tests, user feedback.

7. Practical interview / teaching talking points on AI

1. Start with business use case, not hype.
2. Explain where structured data ends and unstructured data begins.
3. Mention that AI systems still need good ingestion, metadata and quality.
4. Raw data + traceability are critical for AI.
5. Talk about evaluation, monitoring and human review.
6. Discuss security / PII / permissions early.
7. Show trade-offs: accuracy vs latency vs cost.
8. State that data engineering fundamentals remain the base.

8. Common AI + Data Engineering Interview Scenarios

Scenario	How to approach
Build a chatbot on company documents	Ingest docs, parse and chunk, create embeddings, store metadata, retrieval pipeline, permission-aware access, monitor and evaluate.
Keep AI knowledge base updated daily	Incremental ingestion, change-file detection, re-embed changed chunks.
Bad answers from RAG system	Check chunking strateg, metadata, retrieval filters, state data, evaluation metrics.
Duplicate or conflicting docs	Versioning, source ranking, canonicalization, deduplication logic.
Sensitive documents	PII masking, ACL-based retrieval, audit logs, enforce permisions.
High latency or high cost	Use smaller models, batch processing, caching, hybrid retrieval, optimize chunking/embeddings.

Data Engineering Interview Training Notes - Page 5

Schema evolution, data contracts, security & governance, testing, backfills, DR, and cost optimization

1. Schema Evolution

- Data schemas change over time. We need to handle changes without breaking pipelines and consumers.
- **Added columns**: usually backward compatible (old consumers can ignore new columns).
- **Removed columns**: can break consumers. Use deprecation period and communicate early.
- **Renamed columns**: breaking change unless handled with aliases or keeping both (old + new) for a transition period.
- **Type changes**: can be breaking (e.g. int -> string). Use compatible types or create a new column.
- **Backward compatibility**: new data can be read by old consumers (e.g. adding nullable columns).
- **Forward compatibility**: old data can be read by new consumers (e.g. code handles missing columns).
- **Schema-on-read** (e.g. data lake): flexible, but consumers must handle schema changes.
- **Schema enforcement** (e.g. warehouse): strict schema, prevents unexpected changes.
- **Handle new columns** as nullable with default values where possible.
- **Version schemas** (e.g. schema registry) to track changes.
- **Understand consumer impact**: communicate changes, provide deprecation timeline, and test with downstream teams.

2. Data Contracts

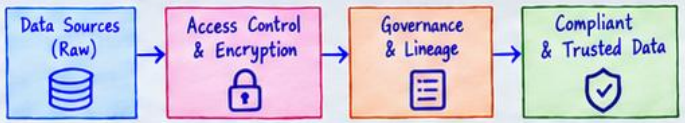
A data contract is an agreement between a data producer and consumer that defines the data, its structure, quality, and SLAs. It helps ensure reliable, predictable, and well-defined data exchange.

Contract Element	Description / Example
Owner	Team or person responsible for the dataset
Schema	Table schema, data types, versioning policy
Field defintions	Name, type, description, valid values (e.g. enums), business meaning
Freshness SLA	Expected data latency (e.g. within 1 hour)
Quality expectations	Null checks, valid ranges, unique constraints, business rules
Allowed nulls	Which fields can be null and under what conditions
Delivery schedule	Frequency (hourly, daily), time window
Deprecation policy	How breaking changes are communicated and timeline for removal
Escalation / contact	Who to contact for issues, on-call or Slack channel

★ **Why data contracts help?**

- Align producer and consumer expectations.
- Reduce breaking changes and improve communication.
- Enable better data quality, monitoring, and governance.
- Support automated validation and documentation.

3. Security, Governance & PII



- **Access control / RBAC**: give appropriate permissions (least privilege).
- **Masking / Tokenization / Encryption**: protect sensitive data at rest and in transit.
- **Row-level and column-level security**: restrict data based on user roles or attributes (e.g. region, department).
- **Auditability**: track who accessed what data and when (access logs).
- **Lineage & catalog**: track data flow, ownership, and transformations.
- **Retention policies**: define how long data is stored and when it is deleted.
- **Compliance mindset**: follow relevant regulations (e.g. GDPR, CCPA, HIPAA).
- **Least privilege**: only give minimal access required for the job.
- **Handling sensitive fields**: PII (name, email, phone, address, etc.) - mask or tokenize and limit access. Avoid exposing raw PII to unnecessary users.

Examples of sensitive fields:

- **Personal identifiers**: name, email, phone, address, national ID.
- **Financial data**: credit card, bank account.
- **Health data**: medical records.
- **Usage data** that can identify a person (e.g. device id, IP address).

4. Pipeline Testing Strategy

Test Type	What it checks	Examples
Unit tests	Test individual functions/ transformations	Parsing logic, UDFs, data transformations
Schema tests	Validate schema and data types	Expected columns, types, nullable fields
Data quality tests	Validate data correctness and business rules	Null checks, range checks, uniqueness, referential checks
Integration tests	Test interaction with external systems	Read/write to storage, APIs, message queues
End-to-end tests	Test full pipeline flow	Ingest -> transform -> load with sample/production-like data
Regression tests	Ensure no unintended changes	Compare row counts, sample data, checksums with previous run
Backfill validation	Verify reprocessed data correctness	Row counts, no duplicates, data consistency with fresh data
Smoke tests	Quick checks after deployment	Pipeline runs successfully, key tables updated

Test data tips:

- Use small, representative datasets.
- Include edge cases (nulls, out-of-range values).
- Mask sensitive data.
- Keep test data versioned.

Automate with CI/CD:

- Run unit and data tests on every commit.
- Use temporary environments.
- Fail the pipeline if critical tests fail.
- Include data quality checks in deployment process (e.g. as a quality gate).

5. Backfills, Reprocessing & Disaster Recovery

- **When to backfill**: fix data issues, onboard new source, logic change, historical data correction, or to rebuild after failures.
- **Idempotent reruns**: design pipelines to be re-runnable without duplicate data (e.g. use upserts/ merge, partition overwrite).
- **Checkpoint resets**: reset checkpoints for streaming jobs to reprocess from a specific point.
- **Replay from offset / watermark**: for streaming systems, start from an earlier offset or timestamp.
- **Partition reprocessing**: reprocess specific date partitions instead of full reload.
- **Rollback / restore**: keep previous versions or backups to rollback bad changes.
- **Recovery point objective (RPO)**: how much data loss is acceptable (e.g. 5 minutes, 1 hour).
- **Recovery time objective (RTO)**: how quickly the system should be restored (e.g. 30 minutes).
- **Safe re-run**: avoid duplicates using idempotent writes, deduplication logic, or MERGE / UPSERT based on business keys.

★ **Best practiess:**

- Keep raw data to enable reprocessing.
- Use versioned data and metadata (e.g. run_id, processing_date).
- Test backfill process in a non-production enviroment.
- Monitor for duplicates and data consistency after reprocessing.

6. Cost Optimization

- **Partition pruning**: filter on partition columns to read only necessary data.
- **Use columnar formats**: Parquet, ORC, etc. for better compression and query performance.
- **Avoid over-partitioning**: too many small partitions can degrade performance and increase metadata cost.
- **Compaction**: combine small files to larger files to improve query efficiency.
- **File sizing**: target optimal file sizes (e.g. 128MB - 1GB).
- **Choose the right compute**: match instance/cluster size to workload.
- **Autoscaling**: automatically scale resources based on load.
- **Scheduling**: turn off non-production clusters when not in use.
- **Caching**: use caching where appropriate (e.g. query result cache).
- **Minimize full reloads**: use incremental loads and change data capture.
- **Monitor storage and compute cost**: set up cost alerts and track usage over time.

7. Production-readiness Checklist

1. Clear owner for the pipeline and data.
2. Defined SLA / expected latency.
3. Monitoring and alerting in place.
4. Data lineage and metadata documented.
5. Documentation (design, runbooks, troubleshooting).
6. Access control and security reviewed.
7. Automated tests (unit, data quality, integration).
8. Data quality checks with clear rules.
9. Backfill / reprocessing plan defined and tested.
10. Cost visibility and optimization considered.

8. Trainer talking points

5. Walk through a backfill scenario step-by-step.
6. Discuss DR strategy and key RPO/RTO.
7. Share practical ways to reduce cost in real systems.
8. Emphasize production-readiness mindset and how you would operationalize it.